

Assignment 21.5

Name: Navya Revuri

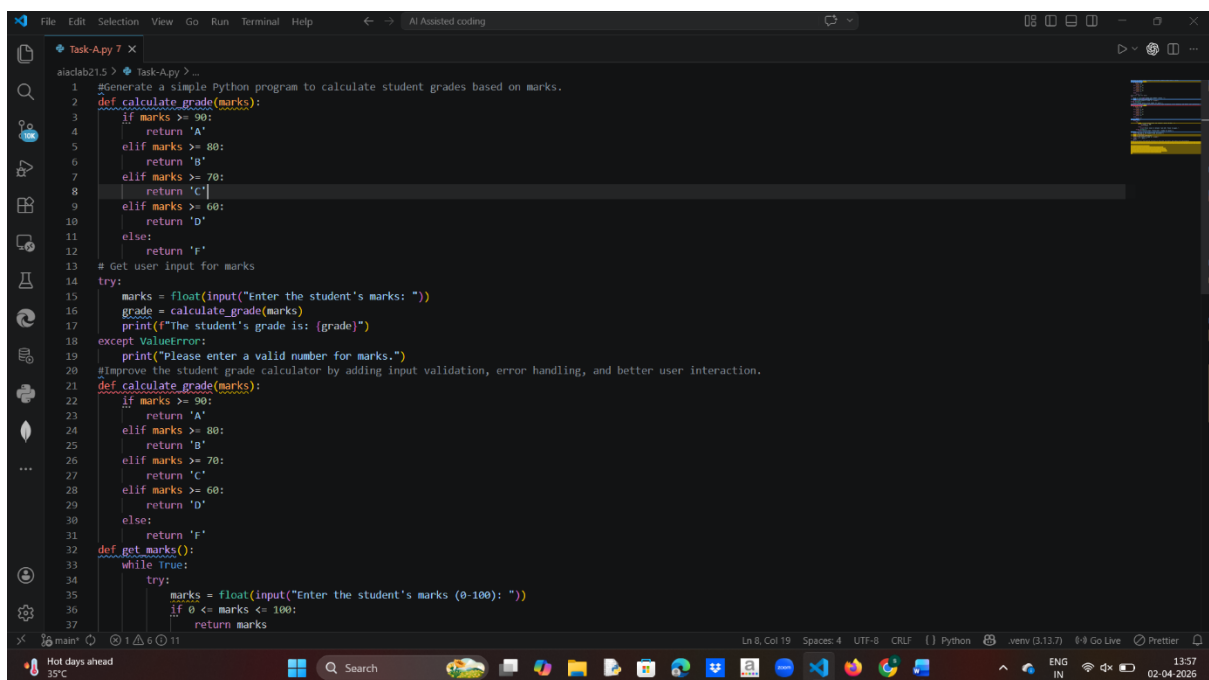
HTNO: 2303A52483

Batch 44

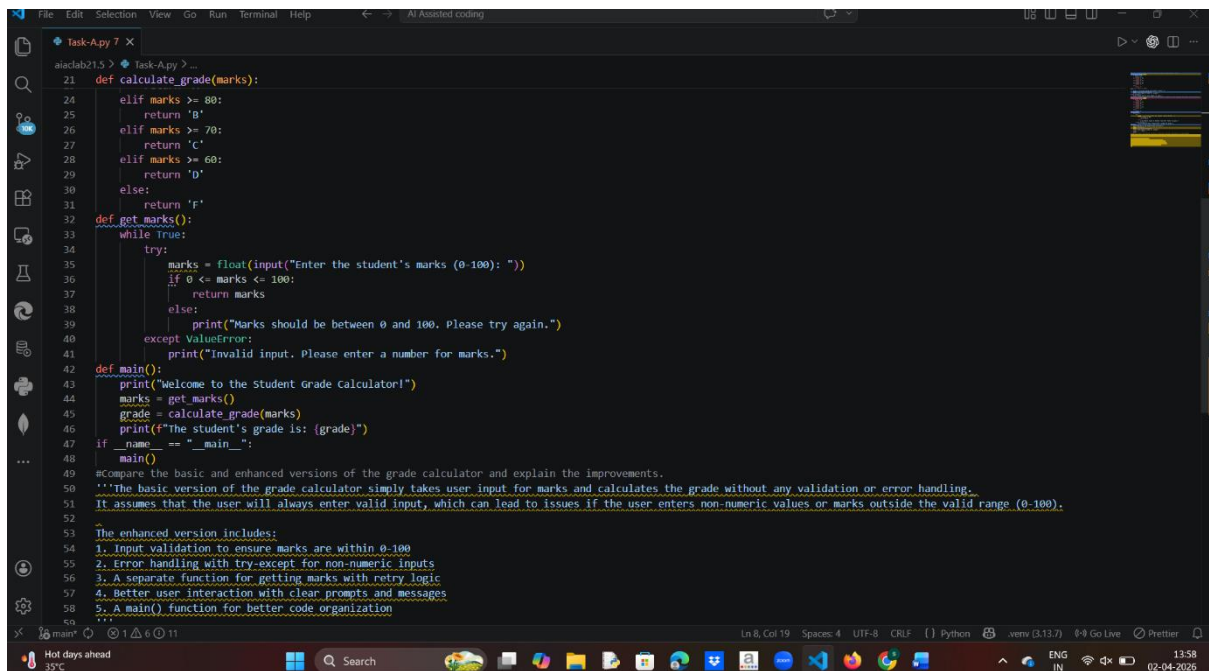
Task A – AI-Based Initial Program Development

Prompt:

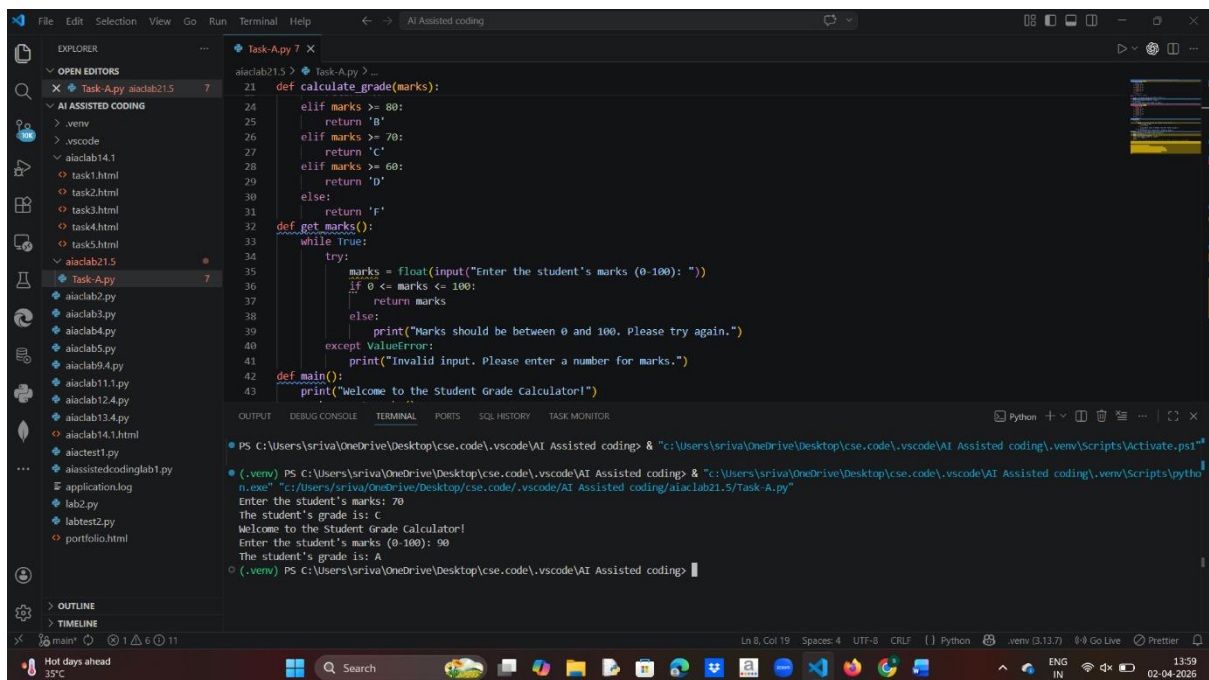
- # Generate a Python program for a student grade calculator.
- # Take marks as input and calculate total and average.
- # Display grade based on average marks.
- # Then enhance the program with input validation.
- # Add features like multiple students and error handling



```
1 #Generate a simple Python program to calculate student grades based on marks.
2 def calculate_grade(marks):
3     if marks >= 90:
4         return 'A'
5     elif marks >= 80:
6         return 'B'
7     elif marks >= 70:
8         return 'C'
9     elif marks >= 60:
10        return 'D'
11    else:
12        return 'F'
13    # Get user input for marks
14    try:
15        marks = float(input("Enter the student's marks: "))
16        grade = calculate_grade(marks)
17        print(f"The student's grade is: {grade}")
18    except ValueError:
19        print("Please enter a valid number for marks.")
20    #Improve the student grade calculator by adding input validation, error handling, and better user interaction.
21    def calculate_grade(marks):
22        if marks >= 90:
23            return 'A'
24        elif marks >= 80:
25            return 'B'
26        elif marks >= 70:
27            return 'C'
28        elif marks >= 60:
29            return 'D'
30        else:
31            return 'F'
32    def get_marks():
33        while True:
34            try:
35                marks = float(input("Enter the student's marks (0-100): "))
36                if 0 <= marks <= 100:
37                    return marks
```



```
21 def calculate_grade(marks):
22     if marks >= 80:
23         return 'A'
24     elif marks >= 70:
25         return 'B'
26     elif marks >= 60:
27         return 'C'
28     elif marks >= 50:
29         return 'D'
30     else:
31         return 'F'
32
33 def get_marks():
34     while True:
35         try:
36             marks = float(input("Enter the student's marks (0-100): "))
37             if 0 <= marks <= 100:
38                 return marks
39             else:
40                 print("Marks should be between 0 and 100. Please try again.")
41         except ValueError:
42             print("Invalid input. Please enter a number for marks.")
43
44 def main():
45     print("Welcome to the Student Grade Calculator!")
46     marks = get_marks()
47     grade = calculate_grade(marks)
48     print(f"The student's grade is: {grade}")
49
50 if __name__ == "__main__":
51     main()
52
53 #Compare the basic and enhanced versions of the grade calculator and explain the improvements.
54 """The basic version of the grade calculator simply takes user input for marks and calculates the grade without any validation or error handling.
55 It assumes that the user will always enter valid input, which can lead to issues if the user enters non-numeric values or marks outside the valid range (0-100).
56
57 The enhanced version includes:
58 1. Input validation to ensure marks are within 0-100
59 2. Error handling with try-except for non-numeric inputs
60 3. A separate function for getting marks with retry logic
61 4. Better user interaction with clear prompts and messages
62 5. A main() function for better code organization
63 """
```



```
PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> & "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\venv\Scripts\activate.ps1"
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> & "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\venv\Scripts\python.exe" "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\aiacalab21.5\Task A.py"
Enter the student's marks: 70
The student's grade is: C
Welcome to the Student Grade Calculator!
Enter the student's marks (0-100): 90
The student's grade is: A
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding>
```

Observation

The initial program performs basic calculations only.

It does not check for invalid inputs like negative marks.

The enhanced version includes validation for correct data entry.

It supports multiple students, making it more practical.

Error handling improves reliability of the program.

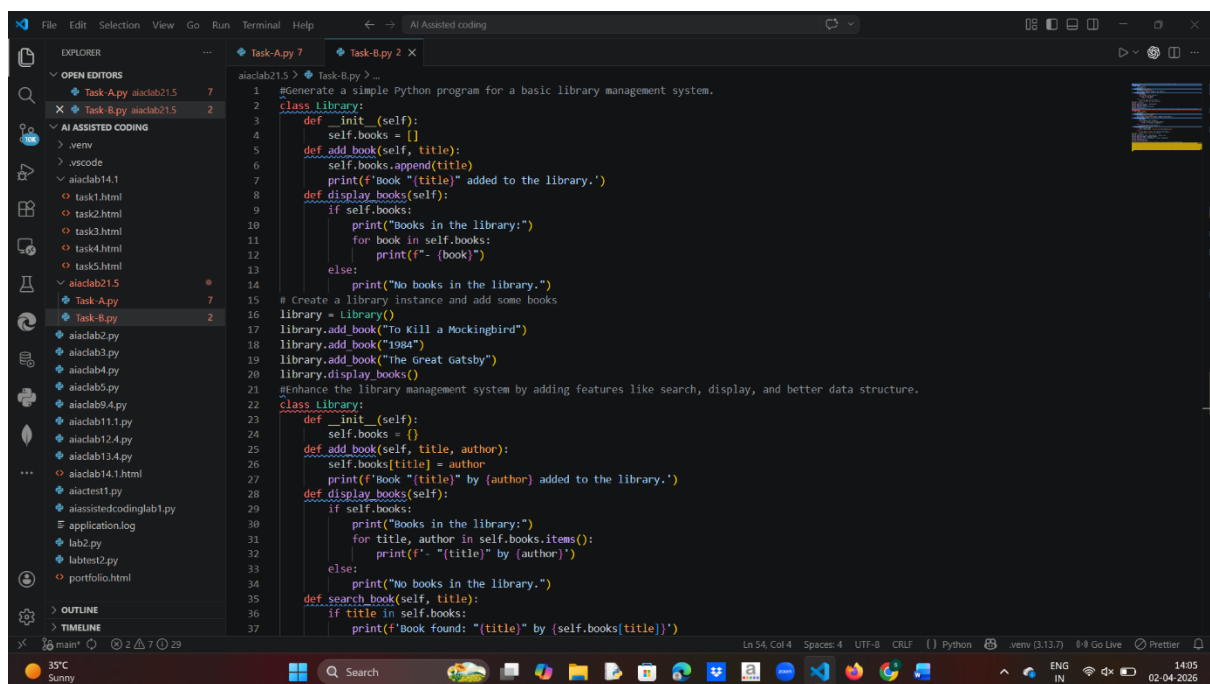
Code structure becomes more modular and readable.

User experience is improved with proper prompts and messages.
Overall, AI significantly improves functionality and robustness.

Task B – AI-Assisted Application Improvement

Prompt:

- # Generate a simple Python library management system.
- # Include features like adding and viewing books.
- # Then enhance it using AI by adding search functionality.
- # Include options for issuing and returning books.
- # Improve code structure and efficiency.



```
1 #Generate a simple Python program for a basic library management system.
2 class Library:
3     def __init__(self):
4         self.books = []
5     def add_book(self, title):
6         self.books.append(title)
7         print(f'Book "{title}" added to the library.')
8     def display_books(self):
9         if self.books:
10             print("Books in the library:")
11             for book in self.books:
12                 print(f'- {book}')
13         else:
14             print("No books in the library.")
15 # Create a library instance and add some books
16 library = Library()
17 library.add_book("To Kill a Mockingbird")
18 library.add_book("1984")
19 library.add_book("The Great Gatsby")
20 library.display_books()
21 #Enhance the library management system by adding features like search, display, and better data structure.
22 class Library:
23     def __init__(self):
24         self.books = {}
25     def add_book(self, title, author):
26         self.books[title] = author
27         print(f'Book "{title}" by {author} added to the library.')
28     def display_books(self):
29         if self.books:
30             print("Books in the library:")
31             for title, author in self.books.items():
32                 print(f'- "{title}" by {author}')
33         else:
34             print("No books in the library.")
35     def search_book(self, title):
36         if title in self.books:
37             print(f'Book found: "{title}" by {self.books[title]}')
```

```
19 library.add_book("The Great Gatsby")
20 library.display_books()
21 #Enhance the library management system by adding features like search, display, and better data structure.
22 class Library:
23     def __init__(self):
24         self.books = {}
25     def add_book(self, title, author):
26         self.books[title] = author
27         print(f'Book "{title}" by {author} added to the library.')
28     def display_books(self):
29         if self.books:
30             print("Books in the library:")
31             for title, author in self.books.items():
32                 print(f'- "{title}" by {author}')
33         else:
34             print("No books in the library.")
35     def search_book(self, title):
36         if title in self.books:
37             print(f'Book found: "{title}" by {self.books[title]}')
38         else:
39             print(f'Book "{title}" not found in the library.')
40 # Create a library instance and add some books
41 library = Library()
42 library.add_book("To Kill a Mockingbird", "Harper Lee")
43 library.add_book("1984", "George Orwell")
44 library.add_book("The Great Gatsby", "F. Scott Fitzgerald")
45 library.display_books()
46 library.search_book("1984")
47 #Analyze how the improved version is better in terms of structure and performance.
48 '''The improved version of the library management system has several enhancements over the basic version:
49 1. Data Structure: The improved version uses a dictionary to store books, allowing for more efficient lookups and better organization of book information.
50 2. Search Functionality: The improved version includes a search function that allows users to find books by title, which is a common feature in library systems.
51 3. Better User Interaction: The improved version provides clearer prompts and messages, enhancing the user experience.
52 4. Code Organization: The improved version is more structured, with separate methods for adding books, displaying books, and searching for books.
53 Overall, the improved version is more functional, user-friendly, and efficient compared to the basic version.'''
54
```

```
PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\venv\Scripts\activate.ps1"
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding> "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\venv\Scripts\python.exe" "C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding\aiacalab21.5\Task-8.py"
Book "To Kill a Mockingbird" added to the library.
Book "1984" added to the library.
Book "The Great Gatsby" added to the library.
Books in the library:
- To Kill a Mockingbird
- 1984
- The Great Gatsby
Book "To Kill a Mockingbird" by Harper Lee added to the library.
Book "1984" by George Orwell added to the library.
Book "The Great Gatsby" by F. Scott Fitzgerald added to the library.
Books in the library:
- "To Kill a Mockingbird" by Harper Lee
- "1984" by George Orwell
- "The Great Gatsby" by F. Scott Fitzgerald
Book found: "1984" by George Orwell
(.venv) PS C:\Users\sriya\OneDrive\Desktop\cse.code\vscode\AI Assisted coding>
```

Observation:

The initial program has limited features and basic structure.

It only supports adding and displaying books.

The improved version includes search and book issue features.

Function-based structure improves code organization.

Efficiency increases with better data handling.

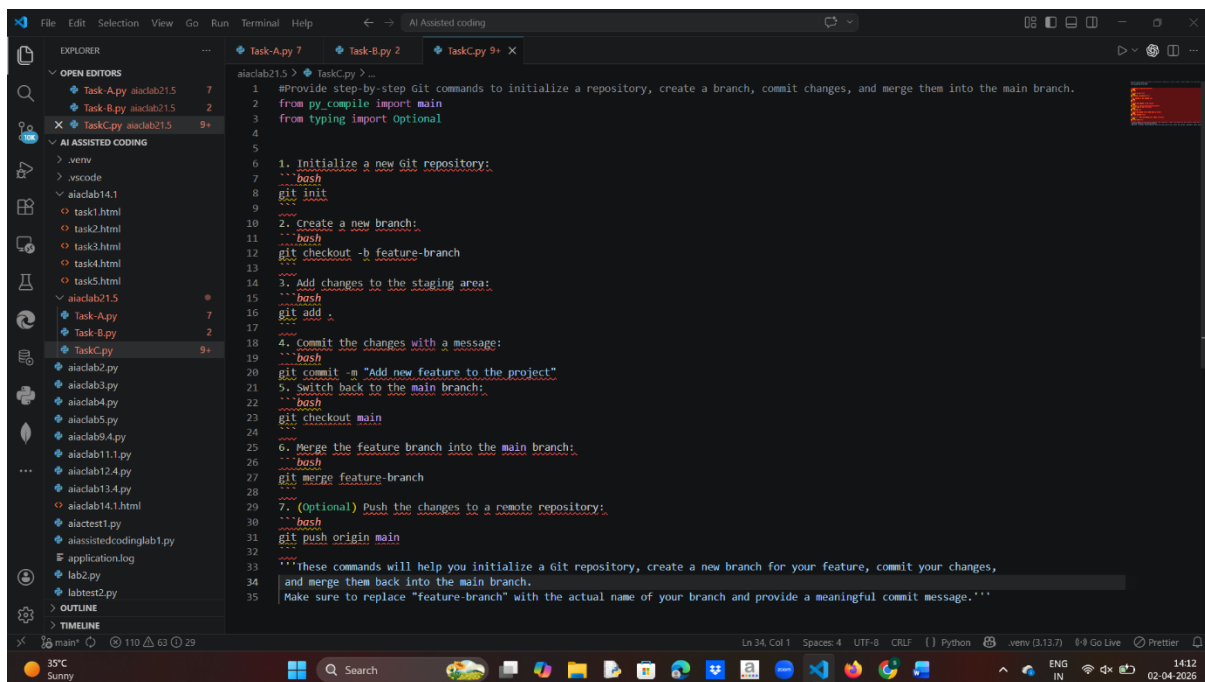
User interaction becomes more dynamic and useful.

The program becomes closer to a real-world application.
AI helps transform a simple script into a complete system.

Task C – Git Workflow Practice

Prompt:

- # Initialize a Git repository for a Python project.
- # Create a new branch for adding new features.
- # Add AI-generated code changes in the branch.
- # Commit the changes with a message.
- # Merge the branch into the main branch.



The screenshot shows a VS Code editor window with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with files like `task1.html`, `task2.html`, `task3.html`, `task4.html`, `task5.html`, `aiacrab21.5`, `Task-A.py`, `Task-B.py`, and `TaskC.py`. The code editor shows the content of `TaskC.py`, which is a Python script that provides step-by-step Git commands to initialize a repository, create a branch, commit changes, and merge them into the main branch. The script is as follows:

```
1 #Provide step-by-step Git commands to initialize a repository, create a branch, commit changes, and merge them into the main branch.
2 from py_compile import main
3 from typing import Optional
4
5
6 1. Initialize a new Git repository:
7 ---bash
8 git init
9 ---
10
11 2. Create a new branch:
12 ---bash
13 git checkout -b feature-branch
14 ---
15
16 3. Add changes to the staging area:
17 ---bash
18 git add .
19 ---
20
21 4. Commit the changes with a message:
22 ---bash
23 git commit -m "Add new feature to the project"
24 ---
25
26 5. Switch back to the main branch:
27 ---bash
28 git checkout main
29 ---
30
31 6. Merge the feature branch into the main branch:
32 ---bash
33 git merge feature-branch
34 ---
35
36 7. (optional) Push the changes to a remote repository:
37 ---bash
38 git push origin main
39 ---
40
41 '''These commands will help you initialize a Git repository, create a new branch for your feature, commit your changes,
42 and merge them back into the main branch.
43 Make sure to replace "feature-branch" with the actual name of your branch and provide a meaningful commit message.'''
```

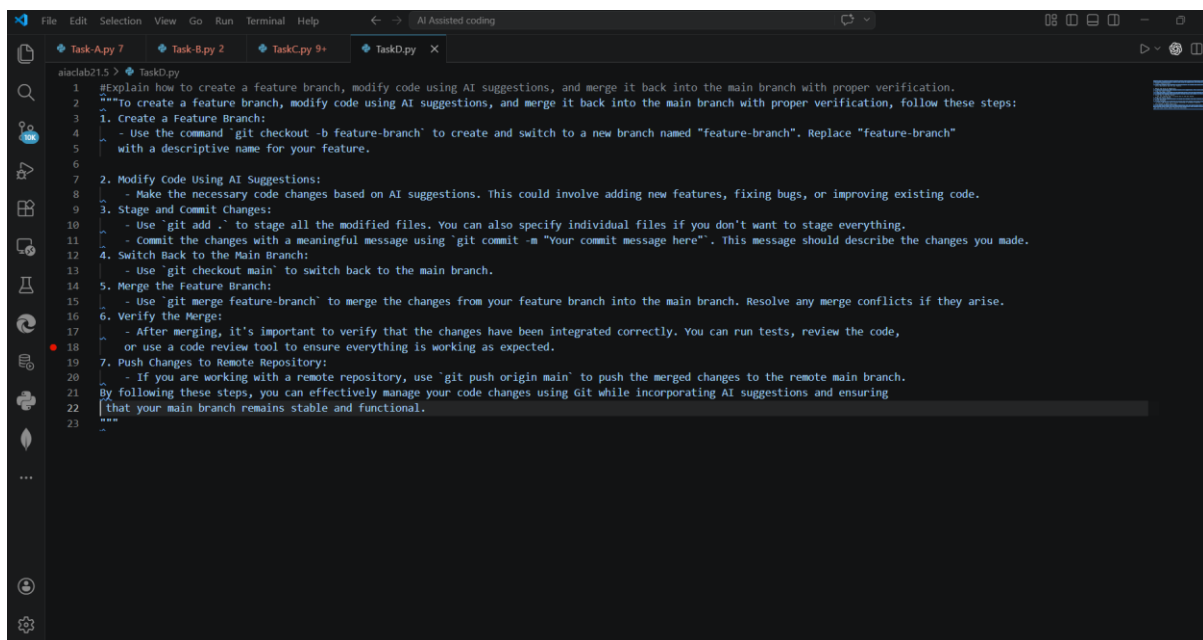
Observation:

- Git initialization sets up version control for the project.
- Branching allows safe development without affecting main code.
- AI-generated changes can be tested separately in branches.
- Commits help track changes with proper history.
- Merging integrates new features into the main branch.
- Conflicts may occur and require resolution.
- Workflow improves collaboration and code management.
- Using Git ensures better organization and rollback capability.

Task D – Branching and Merging with AI Support

Prompt:

- # Develop a basic program in the main branch.
- # Create a feature branch for modifications.
- # Use AI to enhance the program in the feature branch.
- # Merge the updated branch into main.
- # Verify the final program output.



The screenshot shows a code editor window with a dark theme. The editor has a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The main area displays a file named 'TaskD.py' with the following content:

```
1 #Explain how to create a feature branch, modify code using AI suggestions, and merge it back into the main branch with proper verification.
2 """
3 To create a feature branch, modify code using AI suggestions, and merge it back into the main branch with proper verification, follow these steps:
4 1. Create a Feature Branch:
5 - Use the command 'git checkout -b feature-branch' to create and switch to a new branch named "feature-branch". Replace "feature-branch"
6   with a descriptive name for your feature.
7 2. Modify Code Using AI Suggestions:
8 - Make the necessary code changes based on AI suggestions. This could involve adding new features, fixing bugs, or improving existing code.
9 3. Stage and Commit Changes:
10 - Use 'git add .' to stage all the modified files. You can also specify individual files if you don't want to stage everything.
11 - Commit the changes with a meaningful message using 'git commit -m "Your commit message here"'. This message should describe the changes you made.
12 4. Switch Back to the Main Branch:
13 - Use 'git checkout main' to switch back to the main branch.
14 5. Merge the Feature Branch:
15 - Use 'git merge feature-branch' to merge the changes from your feature branch into the main branch. Resolve any merge conflicts if they arise.
16 6. Verify the Merge:
17 - After merging, it's important to verify that the changes have been integrated correctly. You can run tests, review the code,
18   or use a code review tool to ensure everything is working as expected.
19 7. Push Changes to Remote Repository:
20 - If you are working with a remote repository, use 'git push origin main' to push the merged changes to the remote main branch.
21 By following these steps, you can effectively manage your code changes using Git while incorporating AI suggestions and ensuring
22 that your main branch remains stable and functional.
23 """
```

Observation

- The main branch contains the stable version of the program.
- Feature branch allows safe experimentation and improvements.
- AI helps add new features quickly in the branch.
- Merging combines both versions into a single codebase.
- Testing ensures that merged code works correctly.
- Conflicts can occur if both branches modify same lines.
- Proper merging maintains code integrity.
- This workflow supports structured and safe development.

Task E – AI for Commit Message Writing

Prompt:

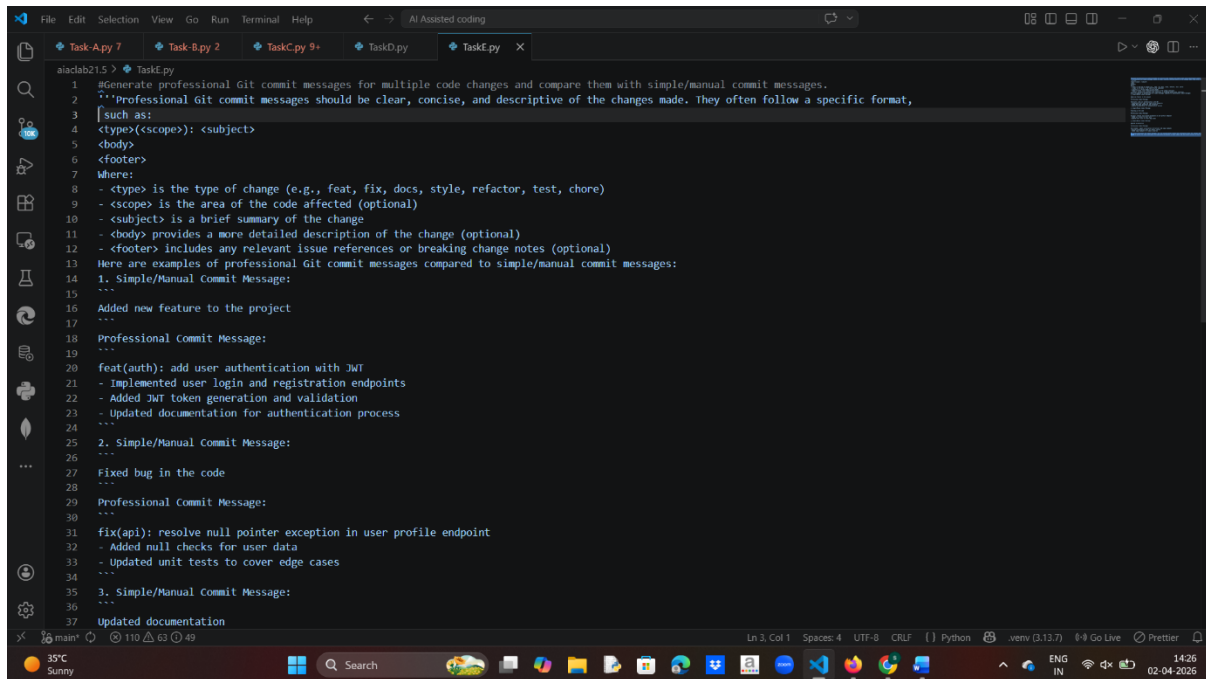
Make multiple changes to a Python program.

Write manual commit messages for each change.

Then use AI to generate commit messages.

Compare both manual and AI-generated messages.

Analyze clarity and usefulness.



```
1 #Generate professional Git commit messages for multiple code changes and compare them with simple/manual commit messages.
2 '''Professional Git commit messages should be clear, concise, and descriptive of the changes made. They often follow a specific format,
3 such as:
4 <type>(<scope>): <subject>
5 <body>
6 <footer>
7 Where:
8 - <type> is the type of change (e.g., feat, fix, docs, style, refactor, test, chore)
9 - <scope> is the area of the code affected (optional)
10 - <subject> is a brief summary of the change
11 - <body> provides a more detailed description of the change (optional)
12 - <footer> includes any relevant issue references or breaking change notes (optional)
13 Here are examples of professional Git commit messages compared to simple/manual commit messages:
14 1. Simple/Manual Commit Message:
15 ...
16 Added new feature to the project
17 ...
18 Professional Commit Message:
19 ...
20 feat(auth): add user authentication with JWT
21 - Implemented user login and registration endpoints
22 - Added JWT token generation and validation
23 - Updated documentation for authentication process
24 ...
25 2. Simple/Manual Commit Message:
26 ...
27 Fixed bug in the code
28 ...
29 Professional Commit Message:
30 ...
31 fix(api): resolve null pointer exception in user profile endpoint
32 - Added null checks for user data
33 - Updated unit tests to cover edge cases
34 ...
35 3. Simple/Manual Commit Message:
36 ...
37 Updated documentation
```

Observation:

Manual commit messages may be unclear or inconsistent.

AI-generated messages are usually more structured and descriptive.

AI highlights key changes effectively in fewer words.

Consistency is better in AI-generated messages.

Manual messages depend on developer experience.

AI helps beginners write professional commit messages.

However, AI may sometimes lack context of deeper changes.

Combining manual understanding with AI gives best results.